sklearn.svm.NuSVR

class `sklearn.svm.`**`NuSVR`**(*, *nu=0.5, C=1.0, kernel='rbf', degree=3, gamma='scale', coef0=0.0, shrinking=True, tol=0.001, cache_size=200, verbose=False, max_iter=- 1*) [source]

Nu Support Vector Regression.

Similar to NuSVC, for regression, uses a parameter nu to control the number of support vectors. However, unlike NuSVC, where nu replaces C, here nu replaces the parameter epsilon of epsilon-SVR.

The implementation is based on libsvm.

Read more in the User Guide.

Parameters:

nu : *float, default=0.5*
An upper bound on the fraction of training errors and a lower bound of the fraction of support vectors. Should be in the interval (0, 1]. By default 0.5 will be taken.

C : *float, default=1.0*
Penalty parameter C of the error term.

kernel : *{'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable, default='rbf'*
Specifies the kernel type to be used in the algorithm. If none is given, 'rbf' will be used. If a callable is given it is used to precompute the kernel matrix.

degree : *int, default=3*
Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : *{'scale', 'auto'} or float, default='scale'*
Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if `gamma='scale'` (default) is passed then it uses 1 / (n_features * X.var()) as value of gamma,
- if 'auto', uses 1 / n_features.

> *Changed in version 0.22:* The default value of `gamma` changed from 'auto' to 'scale'.

coef0 : *float, default=0.0*
Independent term in kernel function. It is only significant in 'poly' and 'sigmoid'.

shrinking : *bool, default=True*
Whether to use the shrinking heuristic. See the User Guide.

tol : *float, default=1e-3*
Tolerance for stopping criterion.

cache_size : *float, default=200*
Specify the size of the kernel cache (in MB).

verbose : *bool, default=False*
Enable verbose output. Note that this setting takes advantage of a per-process runtime setting in libsvm that, if enabled, may not work properly in a multithreaded context.

max_iter : *int, default=-1*
Hard limit on iterations within solver, or -1 for no limit.

Attributes:

class_weight_ : *ndarray of shape (n_classes,)*
Multipliers of parameter C for each class. Computed based on the `class_weight` parameter.

`coef_` : *ndarray of shape (1, n_features)*
Weights assigned to the features when `kernel="linear"`.

dual_coef_ : *ndarray of shape (1, n_SV)*
ts of the support vector in the decision function.

Toggle Menu

fit_status_ : *int*
0 if correctly fitted, 1 otherwise (will raise warning)

intercept_ : *ndarray of shape (1,)*
Constants in decision function.

n_features_in_ : *int*
Number of features seen during fit.

New in version 0.24.

feature_names_in_ : *ndarray of shape (`n_features_in_`,)*
Names of features seen during fit. Defined only when `X` has feature names that are all strings.

New in version 1.0.

n_iter_ : *int*
Number of iterations run by the optimization routine to fit the model.

New in version 1.1.

`n_support_` : *ndarray of shape (n_classes,), dtype=int32*
Number of support vectors for each class.

shape_fit_ : *tuple of int of shape (n_dimensions_of_X,)*
Array dimensions of training vector `X`.

support_ : *ndarray of shape (n_SV,)*
Indices of support vectors.

support_vectors_ : *ndarray of shape (n_SV, n_features)*
Support vectors.

See also:

NuSVC
Support Vector Machine for classification implemented with libsvm with a parameter to control the number of support vectors.

SVR
Epsilon Support Vector Machine for regression implemented with libsvm.

References

1 LIBSVM: A Library for Support Vector Machines

2 Platt, John (1999). "Probabilistic outputs for support vector machines and comparison to regularizedlikelihood methods."

Examples

```
>>> from sklearn.svm import NuSVR
>>> from sklearn.pipeline import make_pipeline
>>> from sklearn.preprocessing import StandardScaler
>>> import numpy as np
>>> n_samples, n_features = 10, 5
>>> np.random.seed(0)
>>> y = np.random.randn(n_samples)
>>> X = np.random.randn(n_samples, n_features)
>>> regr = make_pipeline(StandardScaler(), NuSVR(C=1.0, nu=0.1))
>>> regr.fit(X, y)
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('nusvr', NuSVR(nu=0.1))])
```

Methods

`fit`(X, y[, sample_weight])	Fit the SVM model according to the given training data.
`get_params`([deep])	Get parameters for this estimator.
`predict`(X)	Perform regression on samples in X.
`score`(X, y[, sample_weight])	Return the coefficient of determination of the prediction.
`set_params`(**params)	Set the parameters of this estimator.

Toggle Menu

property **`coef_`**

Weights assigned to the features when `kernel="linear"`.

Returns:

ndarray of shape (n_features, n_classes)

`fit`(*X*, *y*, *sample_weight=None*) [source]

Fit the SVM model according to the given training data.

Parameters:

X : *{array-like, sparse matrix} of shape (n_samples, n_features) or (n_samples, n_samples)*

Training vectors, where `n_samples` is the number of samples and `n_features` is the number of features. For kernel="precomputed", the expected shape of X is (n_samples, n_samples).

y : *array-like of shape (n_samples,)*

Target values (class labels in classification, real numbers in regression).

sample_weight : *array-like of shape (n_samples,), default=None*

Per-sample weights. Rescale C per sample. Higher weights force the classifier to put more emphasis on these points.

Returns:

self : *object*

Fitted estimator.

Notes

If X and y are not C-ordered and contiguous arrays of np.float64 and X is not a scipy.sparse.csr_matrix, X and/or y may be copied.

If X is a dense array, then the other methods will not support sparse matrices as input.

`get_params`(*deep=True*) [source]

Get parameters for this estimator.

Parameters:

deep : *bool, default=True*

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Returns:

params : *dict*

Parameter names mapped to their values.

property **`n_support_`**

Number of support vectors for each class.

`predict`(*X*) [source]

Perform regression on samples in X.

For an one-class model, +1 (inlier) or -1 (outlier) is returned.

Parameters:

X : *{array-like, sparse matrix} of shape (n_samples, n_features)*

For kernel="precomputed", the expected shape of X is (n_samples_test, n_samples_train).

Returns:

y_pred : *ndarray of shape (n_samples,)*

The predicted values.

Toggle Menu

score(*X*, *y*, *sample_weight=None*) [source]

Return the coefficient of determination of the prediction.

The coefficient of determination R^2 is defined as $(1 - \frac{u}{v})$, where u is the residual sum of squares `((y_true - y_pred)** 2).sum()` and v is the total sum of squares `((y_true - y_true.mean()) ** 2).sum()`. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). A constant model that always predicts the expected value of `y`, disregarding the input features, would get a R^2 score of 0.0.

Parameters:

X : ***array-like of shape (n_samples, n_features)***
Test samples. For some estimators this may be a precomputed kernel matrix or a list of generic objects instead with shape `(n_samples, n_samples_fitted)`, where `n_samples_fitted` is the number of samples used in the fitting for the estimator.

y : ***array-like of shape (n_samples,) or (n_samples, n_outputs)***
True values for `X`.

sample_weight : ***array-like of shape (n_samples,), default=None***
Sample weights.

Returns:

score : ***float***
R^2 of `self.predict(X)` wrt. `y`.

Notes

The R^2 score used when calling `score` on a regressor uses `multioutput='uniform_average'` from version 0.23 to keep consistent with default value of `r2_score`. This influences the `score` method of all the multioutput regressors (except for `MultiOutputRegressor`).

set_params(***params*) [source]

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as `Pipeline`). The latter have parameters of the form `<component>__<parameter>` so that it's possible to update each component of a nested object.

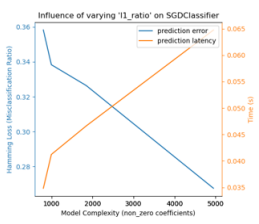
Parameters:

****params :** ***dict***
Estimator parameters.

Returns:

self : ***estimator instance***
Estimator instance.

Examples using `sklearn.svm.NuSVR`



Model Complexity Influence

© 2007 - 2022, scikit-learn developers (BSD License). Show this page source

Toggle Menu